

Code Hardener

ATTESTATION CERTIFICATE



bulletproof-conductor-data-pipeline

Scan 472764fb | Branch: main | Jul 24, 2026

Attestation ID

44a4ee72-c56d-4e9c-b62f-...

Type

in-toto

Subject

bulletproof-conductor-data-pipeline

Created

Jul 24, 2026, 5:09 PM

Subject Digest

sha256:d658fd3ccb5e2dc8c165306bd20b9044fb6a2495e75dab7be9e7f9ac...

Signature Status

CRYPTOGRAPHICALLY SIGNED

Algorithm: Ed25519 (Local Signing Key)

Signature

FCVZ2+1r1iHfTYhAiIwmeGAAuW28kXx5YhwbfbxQLHF1gypfJY86d/T8X0bUGafx
Ls1/TS/6t0LVgAQ1FCUNCA==

This attestation certifies the security scan was performed and results were recorded with cryptographic integrity.

Scan Report — bulletproof-conductor-data-pipeline

Branch: main | Status: completed | Jul 24, 2026

928 / 1000 Score — Quality: excellent

Scan Details

Profile: standard
Duration: 17s
Started: Jul 24, 2026, 5:08 PM
Completed: Jul 24, 2026, 5:09 PM
Files Analyzed: 114
Scanners Run: 11 of 12
Open Findings: 23
Resolved / Suppressed:0
Total Detected: 23
Codebase: Python: 76 files | Config: 14 files | Markdown: 6 files | Shell: 2 files | Docker: 1 files

0 Critical 0 High 12 Medium 3 Low 8 Info

Scanner Results

12 tools executed — 7 passed | 4 with open findings | 1 not applicable | 0 errors

Scanner	Category	Outcome	Findings	Duration	Files Scanned
trivy	SCA	FAIL	1	2s	3
gitleaks	Secrets	PASS	0	0s	114
opengrep	SAST	FAIL	12	6s	78
checkov	IaC	PASS	0	3s	15
grype	SCA	PASS	0	3s	114
syft	SBOM	FAIL	2	1s	2

package-validator	AI Code Quality	PASS	0	0s	1
oxlint	Code Quality	N/A	0	0s	78
ruff	Code Quality	PASS	0	0s	76
actionlint	CI/CD Security	PASS	0	0s	14
jscpd	Code Quality	PASS	0	0s	76
typos	Code Quality	FAIL	8	0s	114

Scanner Evidence & Audit Trail

trivy (SCA)

FINDINGS

Scan Scope: Scanned 3 target(s): kubernetes, dockerfile

Files Analyzed: 3

Rules Checked: 4

Checks Performed:

- Vulnerability scanning
- Misconfiguration detection
- Secret detection
- License compliance

Standards: CVE/NVD, OWASP Top 10 (A06), CIS Benchmarks, NIST SP 800-53

Methodology:

Scans filesystem for package manifests and lock files, matches installed packages against vulnerability databases (NVD, GitHub Advisory, vendor-specific). Also checks for misconfigurations and secrets using built-in rules.

Targets Analyzed:

agents/classification-patterns.yaml, masking_engine/Dockerfile, Loose File License(s)

gitleaks (Secrets)

PASS

Scan Scope: Working directory scanned for 150+ secret patterns with entropy analysis

Configuration: Custom gitleaks.toml

Files Analyzed: 114

Rules Checked: 150

Checks Performed:

- AWS credential detection
- API key detection
- Private key detection
- Generic secret patterns
- High-entropy string analysis
- Token detection

Standards: CWE-798, CWE-259, OWASP Top 10 (A07)

Methodology:

Regex-based pattern matching against 150+ secret patterns with entropy analysis. Scans current file contents and optionally git commit history. Applies allowlists and custom rules to reduce false positives. Secrets are redacted in output.

opengrep (SAST)

FINDINGS

Scan Scope: Multi-language SAST with auto, security-audit, owasp-top-ten, cwe-top-25, r2c-security-audit, and secrets rulesets

Configuration: Custom semgrep-rules.yml + default rulesets

Files Analyzed: 78

Rules Checked: 3

Checks Performed:

- OWASP Top 10 patterns
- Security audit rules
- Secrets detection
- Injection patterns
- XSS patterns
- Insecure crypto usage

Standards: OWASP Top 10, CWE Top 25, SANS Top 25

Methodology:

Pattern-based static analysis using Semgrep rule engine with auto, security-audit, owasp-top-ten, and secrets rulesets. Matches AST patterns against known vulnerability signatures.

checkov (IaC)**PASS**

Scan Scope: IaC security analysis — 0 files, 0 checks (0 passed, 0 failed), frameworks: auto-detected

Configuration: Custom .checkov.yaml

Files Analyzed: 15

Checks Performed:

- Terraform misconfigurations
- CloudFormation template issues
- Kubernetes manifest hardening
- Dockerfile best practices
- Helm chart security
- GitHub Actions security
- GitLab CI security
- Secrets in IaC files

Standards: CIS Benchmarks, NIST SP 800-53, SOC 2, PCI-DSS, HIPAA

Methodology:

Graph-based static analysis of infrastructure-as-code files. Parses IaC definitions, builds resource graphs, and checks 1000+ built-in policies covering security, networking, encryption, and compliance.

grype (SCA)**PASS**

Scan Scope: SCA analysis: 0 unique CVEs across 0 vulnerable packages (filesystem)

Configuration: Grype with --by-cve deduplication and --add-cpes-if-none

Files Analyzed: 114

Checks Performed:

- CVE vulnerability matching
- Dependency version analysis
- CPE-based matching
- Advisory cross-referencing
- Fix availability detection

Standards: CVE/NVD, OWASP Top 10 (A06), GHSA

Methodology:

Generates SBOM using Syft, then matches all identified packages against vulnerability databases (NVD, GitHub Advisories, OS vendor databases). Reports CVEs with severity, fix versions, and references.

syft (SBOM)**FINDINGS**

Scan Scope: SBOM of 2 packages across 0 language(s): N/A

Configuration: Syft with JSON + CycloneDX output for SBOM standards compliance

Files Analyzed: 2

Rules Checked: 4

Checks Performed:

- SBOM generation (Syft JSON + CycloneDX)
- License compliance analysis

- Copyleft license detection (AGPL, GPL, MPL)
- Weak copyleft classification (LGPL)
- Unknown license flagging
- Package URL (PURL) generation
- CPE generation

Standards: NTIA SBOM Minimum Elements, CycloneDX, SPDX, Executive Order 14028

Methodology:

Catalogs software packages by parsing package manifests, lock files, and installed package databases. Generates standardized SBOM in CycloneDX or SPDX format for supply chain visibility.

package-validator (AI Code Quality)

PASS

Scan Scope: Validated 9 packages across 1 dependency file(s)

Configuration: Concurrency: 10, Timeout: 5000ms per request

Files Analyzed: 1

Rules Checked: 1

Checks Performed:

- npm registry existence validation
- PyPI registry existence validation
- Go module proxy existence validation
- Slopsquatting risk detection
- Hallucinated dependency identification

Standards: CWE-829, OWASP A08:2021, SLSA Supply Chain Security

Methodology:

Parses dependency manifests and validates each package exists in its respective registry (npm, PyPI, Go proxy) via HTTP HEAD requests. Non-existent packages are flagged as hallucinated — a common AI code generation artifact that creates supply chain attack vectors.

oxlint (Code Quality)

N/A

Scan Scope: JavaScript (.js, .jsx) and TypeScript (.ts, .tsx) source files

Files Analyzed: 78

Checks Performed:

- Code correctness checks
- Suspicious code detection
- Performance anti-patterns
- Import/export validation

Standards: CWE, ESLint Core Rules

Methodology:

Rust-based linter that performs AST analysis of JS/TS files at high speed. Detects correctness issues, suspicious patterns, and performance anti-patterns.

ruff (Code Quality)

PASS

Scan Scope: Python linting and security analysis via Ruff

Configuration: Default Ruff rules

Files Analyzed: 76

Checks Performed:

- Pyflakes error detection
- pycodestyle enforcement
- flake8-bandit security checks
- isort import ordering
- pep8-naming conventions
- pyupgrade modernization
- flake8-bugbear bug detection
- flake8-comprehensions optimization

Standards: PEP 8, PEP 257, CWE, OWASP Top 10

Methodology:

Rust-based Python linter implementing 800+ rules from flake8, pylint, isort, and others. 10-100x faster than traditional Python linters.

actionlint (CI/CD Security)

PASS

Scan Scope: .github/workflows/*.yml files

Files Analyzed: 14

Checks Performed:

- Workflow syntax validation
- Action version pinning checks
- Shell script validation
- Expression type checking
- Permissions analysis

Standards: GitHub Actions Security Best Practices, CWE-78

Methodology:

Static analysis of GitHub Actions workflow files. Validates syntax, checks for unpinned action versions, analyzes shell scripts for errors, and verifies expression types.

jscpd (Code Quality)

PASS

Scan Scope: All source files across supported languages

Files Analyzed: 76

Checks Performed:

- Cross-file duplicate detection
- Code clone identification
- Duplication percentage calculation

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Token-based analysis to detect copy-pasted code blocks across files and languages. Reports duplication percentage and identifies specific cloned regions.

typos (Code Quality)

FINDINGS

Scan Scope: All files in project directory

Files Analyzed: 114

Checks Performed:

- Source code spelling analysis
- Identifier typo detection
- Comment typo detection
- String literal typo detection

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Fast, low-false-positive spell checker designed for source code. Checks identifiers, strings, and comments against a curated dictionary.

Open Findings (23)

opengrep

SAST | 12 findings | Multi-language static analysis for security vulnerabilities

Severity	Title	File	CWE / Rule
----------	-------	------	------------

MEDIUM python-logger-credential-disclosure .../tools/credential_resolver.py:100 CWE-532: Insertion of
Detected a python logger call with a potential hardcoded secret "Credential not found: variable=%s source=%s error=Resolution failed" being logged.
This may lead to secret credentials being exposed. Make sure that the logger is not logging sensitive information.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected a python logger call with a potential
hardcoded secret "Credential not found: variable=%s source=%s error=Resolution failed" being logged. This may lead to
secret credentials being exposed. M

OWASP: A09:2021 - Security Logging and Monitoring Failures | Rule: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-disclosure

MEDIUM formatted-sql-query ...et/quality/duckdb_executor.py:200 CWE-89: Improper Neutr
Detected possible formatted SQL query. Use parameterized queries instead.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected possible formatted SQL query. Use
parameterized queries instead.

OWASP: A01:2017 - Injection | Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query

MEDIUM formatted-sql-query ...et/quality/duckdb_executor.py:129 CWE-89: Improper Neutr
Detected possible formatted SQL query. Use parameterized queries instead.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected possible formatted SQL query. Use
parameterized queries instead.

OWASP: A01:2017 - Injection | Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query

MEDIUM formatted-sql-query ...et/quality/duckdb_executor.py:101 CWE-89: Improper Neutr
Detected possible formatted SQL query. Use parameterized queries instead.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected possible formatted SQL query. Use
parameterized queries instead.

OWASP: A01:2017 - Injection | Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query

MEDIUM formatted-sql-query ...et/quality/duckdb_executor.py:75 CWE-89: Improper Neutr
Detected possible formatted SQL query. Use parameterized queries instead.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected possible formatted SQL query. Use
parameterized queries instead.

OWASP: A01:2017 - Injection | Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query

MEDIUM formatted-sql-query ...et/quality/duckdb_executor.py:39 CWE-89: Improper Neutr
Detected possible formatted SQL query. Use parameterized queries instead.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected possible formatted SQL query. Use
parameterized queries instead.

OWASP: A01:2017 - Injection | Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query

MEDIUM python-logger-credential-disclosure ...arget/gates/human_approval.py:456 CWE-532: Insertion of
Detected a python logger call with a potential hardcoded secret "AUDIT: token=%s ip=%s decision=%s" being logged. This may lead to secret credentials being exposed. Make sure that the logger is not logging sensitive information.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected a python logger call with a potential hardcoded secret "AUDIT: token=%s ip=%s decision=%s" being logged. This may lead to secret credentials being exposed. Make sure that the logger is not lo

OWASP: A09:2021 - Security Logging and Monitoring Failures | Rule: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-disclosure

MEDIUM python-logger-credential-disclosure ...arget/gates/human_approval.py:344 CWE-532: Insertion of
Detected a python logger call with a potential hardcoded secret "Token %s timed out -- automatic rejection for pipeline=%s" being logged. This may lead to secret credentials being exposed. Make sure that the logger is not logging sensitive information.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected a python logger call with a potential hardcoded secret "Token %s timed out -- automatic rejection for pipeline=%s" being logged. This may lead to secret credentials being exposed. Make sure t

OWASP: A09:2021 - Security Logging and Monitoring Failures | Rule: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-disclosure

MEDIUM python-logger-credential-disclosure ...arget/gates/human_approval.py:307 CWE-532: Insertion of
Detected a python logger call with a potential hardcoded secret "Approval decision for token %s: %s (pipeline=%s)" being logged. This may lead to secret credentials being exposed. Make sure that the logger is not logging sensitive information.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected a python logger call with a potential hardcoded secret "Approval decision for token %s: %s (pipeline=%s)" being logged. This may lead to secret credentials being exposed. Make sure that the l

OWASP: A09:2021 - Security Logging and Monitoring Failures | Rule: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-disclosure

MEDIUM python-logger-credential-disclosure ...arget/gates/human_approval.py:225 CWE-532: Insertion of
Detected a python logger call with a potential hardcoded secret "Generated approval token %s for pipeline=%s contract=%s (expires in %ds)" being logged. This may lead to secret credentials being exposed. Make sure that the logger is not logging sensitive information.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: Detected a python logger call with a potential hardcoded secret "Generated approval token %s for pipeline=%s contract=%s (expires in %ds)" being logged. This may lead to secret credentials being expos

OWASP: A09:2021 - Security Logging and Monitoring Failures | Rule: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-disclosure

MEDIUM github-actions-mutable-action-tag ...rget/github/workflows/ci.yml:12 CWE-1357: Reliance on
GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-github-action compromises. Pin the reference to a full 40-character commit SHA instead, e.g. `uses: actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608`.

requires login

Fix: Review this finding and apply the appropriate fix based on the description: GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-gi

OWASP: A08:2021 - Software and Data Integrity Failures | Rule: yaml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag

MEDIUM github-actions-mutable-action-tag ...rget/.github/workflows/ci.yml:11 CWE-1357: Reliance on GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-github-action compromises. Pin the reference to a full 40-character commit SHA instead, e.g. `uses: actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608`.

```
requires: login
```

Fix: Review this finding and apply the appropriate fix based on the description: GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-gi

OWASP: A08:2021 - Software and Data Integrity Failures | Rule: yaml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag

syft

SBOM | 2 findings | Software bill of materials generator for supply chain visibility

Severity	Title	File	CWE / Rule
LOW	Unknown License: actions/setup-python@v5 Package actions/setup-python has no license information. This may indicate a compliance risk.	/.github/workflows/ci.yml	SBOM-LICENSE-UNKNOWN

```
Package: actions/setup-python
Version: v5
Type: github-action
```

Fix: Verify the license for this package before using in production.

OWASP: A08:2021-Software and Data Integrity Failures

LOW	Unknown License: actions/checkout@v4 Package actions/checkout has no license information. This may indicate a compliance risk.	/.github/workflows/ci.yml	SBOM-LICENSE-UNKNOWN
-----	---	---------------------------	----------------------

```
Package: actions/checkout
Version: v4
Type: github-action
```

Fix: Verify the license for this package before using in production.

OWASP: A08:2021-Software and Data Integrity Failures

trivy

SCA | 1 finding | Container image and filesystem vulnerability scanner

Severity	Title	File	CWE / Rule
LOW	License Compliance: Apache-2.0 in Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy.	LICENSE	LICENSE-Apache-2.0

```
Package:
License: Apache-2.0
Category: notice
```

Fix: Review this finding and apply the appropriate fix based on the description: Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy.

OWASP: A06:2021-Vulnerable and Outdated Components

Severity	Title	File	CWE / Rule
INFO	Typo: "Hashi" should be "Hash" Found typo "Hashi" in typo. Suggested correction: Hash. Fix: Replace "Hashi" with "Hash".	tools/credential_resolver.py:113	TYPOS-SPELLING
INFO	Typo: "Hashi" should be "Hash" Found typo "Hashi" in typo. Suggested correction: Hash. Fix: Replace "Hashi" with "Hash".	tools/credential_resolver.py:5	TYPOS-SPELLING
INFO	Typo: "strat" should be "start, strata" Found typo "strat" in typo. Suggested corrections: start, strata. Fix: Replace "strat" with "start, strata".	tools/data_mask.py:138	TYPOS-SPELLING
INFO	Typo: "strat" should be "start, strata" Found typo "strat" in typo. Suggested corrections: start, strata. Fix: Replace "strat" with "start, strata".	tools/data_mask.py:136	TYPOS-SPELLING
INFO	Typo: "strat" should be "start, strata" Found typo "strat" in typo. Suggested corrections: start, strata. Fix: Replace "strat" with "start, strata".	tools/data_mask.py:134	TYPOS-SPELLING
INFO	Typo: "strat" should be "start, strata" Found typo "strat" in typo. Suggested corrections: start, strata. Fix: Replace "strat" with "start, strata".	tools/data_mask.py:132	TYPOS-SPELLING
INFO	Typo: "strat" should be "start, strata" Found typo "strat" in typo. Suggested corrections: start, strata. Fix: Replace "strat" with "start, strata".	tools/data_mask.py:131	TYPOS-SPELLING
INFO	Typo: "Bypas" should be "Bypass" Found typo "Bypas" in typo. Suggested correction: Bypass. Fix: Replace "Bypas" with "Bypass".	tests/test_steward_gate.py:326	TYPOS-SPELLING

Codebase Coverage (114 files analyzed)

This scan analyzed 114 files using 11 security tools across 8 testing categories. Each scanner targets files relevant to its analysis domain (e.g., SAST scanners analyze source code, SCA scanners analyze dependency manifests).

Files by Language:

Python: 76 files	Config: 14 files
Markdown: 6 files	Shell: 2 files
Docker: 1 files	

Files by Extension:

.py: 76	.other: 12	.yaml: 6
.json: 5	.txt: 4	.yml: 2
.sh: 2	.gitignore: 2	.md: 2
.tag: 1	.dockerfile: 1	.ini: 1

Per-Scanner File Coverage:

Scanner	Category	Files Analyzed	Scope
trivy	SCA	3	Scanned 3 target(s): kubernetes, dockerfile
gitleaks	Secrets	114	Working directory scanned for 150+ secret p...
opengrep	SAST	78	Multi-language SAST with auto, security-aud...
checkov	IaC	15	IaC security analysis — 0 files, 0 checks (...)
grype	SCA	114	SCA analysis: 0 unique CVEs across 0 vulner...
syft	SBOM	2	SBOM of 2 packages across 0 language(s): N/A
package-validator	AI Code Quality	1	Validated 9 packages across 1 dependency fi...
oxlint	Code Quality	78	JavaScript (.js, .jsx) and TypeScript (.ts,...)
ruff	Code Quality	76	Python linting and security analysis via Ruff
actionlint	CI/CD Security	14	.github/workflows/*.yaml files
jscpd	Code Quality	76	All source files across supported languages
typos	Code Quality	114	All files in project directory

Files with Findings (9):

File Path	Findings	Scanners
/scan-target/quality/duckdb_executor.py	5	opengrep
tools/data_mask.py	5	typos
/scan-target/gates/human_approval.py	4	opengrep
./github/workflows/ci.yml	2	syft
/scan-target/./github/workflows/ci.yml	2	opengrep
tools/credential_resolver.py	2	typos
tests/test_steward_gate.py	1	typos
/scan-target/tools/credential_resolver.py	1	opengrep
LICENSE	1	trivy

Software Bill of Materials (2 packages)

Package types: github-action: 2

Top licenses: Unknown (2)

Package	Version	Type	Language	License
actions/checkout	v4	github-actio	-	Unknown
actions/setup-python	v5	github-actio	-	Unknown